MATERIA

INF761

Desarrollo de Videojuegos

Guía de Laboratorio 8

Platformer IV: Juego Completo — UI, Menús y Flujo de Juego

Carrera: Ingeniería Informática

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Semestre: I-2026

Universidad: Universidad Autónoma Tomás Frías (UATF)

Objetivos de Aprendizaje

1. Construir un sistema de UI completo con Canvas, HUD (vidas, puntaje), menú principal y pantalla de Game Over utilizando TextMeshPro y componentes de UI de Unity.
2. Implementar la gestión de escenas con SceneManager para transiciones entre menú principal, niveles de juego y pantalla de fin.
3. Utilizar DontDestroyOnLoad para persistir el GameManager entre escenas y PlayerPrefs para guardar el puntaje más alto.
4. Crear un sistema de checkpoints que actualice dinámicamente el punto de reaparición del jugador y completar un juego de plataformas funcional de principio a fin.

Prerrequisitos

- Guías 5, 6 y 7 completadas: proyecto PlatformerGame con nivel de tilemap, personaje animado con movimiento pulido, enemigos, coleccionables y sistema de vidas.
- Unity 6 LTS con el proyecto PlatformerGame abierto.

1. Marco Teórico

1.1 Sistema de UI en Unity: Canvas

El sistema de UI de Unity se basa en el Canvas, un componente que define el espacio donde se renderizan los elementos de interfaz (botones, textos, imágenes). Todo elemento de UI debe ser hijo de un Canvas. Unity ofrece tres modos de renderizado:

Modo	Descripción
Screen Space – Overlay	El Canvas se dibuja sobre todo (predeterminado). Ideal para HUD, menús. No se ve afectado por la cámara.
Screen Space – Camera	El Canvas se renderiza a una distancia fija de la cámara. Permite efectos 3D sobre la UI.
World Space	El Canvas existe en el mundo 3D como un GameObject más. Útil para barras de vida sobre enemigos.

Canvas Scaler

El componente Canvas Scaler controla cómo se escala la UI en diferentes resoluciones.

Para juegos 2D, usar UI Scale Mode: Scale With Screen Size con Reference Resolution 1920x1080 y Match: 0.5 (balance entre ancho y alto).

Sin esto, la UI se verá diferente en cada resolución de pantalla.

1.2 TextMeshPro (TMP)

TextMeshPro es el sistema de renderizado de texto profesional de Unity. Reemplaza al componente Text legacy con renderizado de mayor calidad basado en Signed Distance Fields (SDF). Los textos TMP se ven nítidos a cualquier tamaño y soportan Rich Text, outlines, sombras y efectos avanzados. En código, se accede mediante el namespace TMPro.

TMP Essential Resources

La primera vez que se agrega un componente TextMeshPro, Unity muestra un diálogo para importar TMP Essential Resources. Es obligatorio importarlos; sin ellos, los textos no se renderizan correctamente. También se puede importar manualmente desde Window > TextMeshPro > Import TMP Essential Resources.

1.3 Gestión de Escenas: SceneManager

Unity organiza el juego en escenas. Cada escena es un archivo .unity que contiene una jerarquía de GameObjects. El SceneManager (namespace UnityEngine.SceneManagement) permite cargar, descargar y transicionar entre escenas en tiempo de ejecución.

```
C# — Uso básico de SceneManager
using UnityEngine.SceneManagement;

// Cargar escena por nombre
SceneManager.LoadScene("MainMenu");

// Recargar la escena actual
SceneManager.LoadScene(
    SceneManager.GetActiveScene().name
);

// Cargar escena por índice (Build Settings)
SceneManager.LoadScene(1);
```

1.4 DontDestroyOnLoad

Por defecto, al cargar una nueva escena, todos los GameObjects de la escena anterior se destruyen. DontDestroyOnLoad marca un GameObject para que sobreviva entre transiciones de escena. Es esencial para el GameManager (que mantiene puntaje y vidas), el AudioManager, o cualquier sistema que deba persistir durante todo el juego.

Precaución con DontDestroyOnLoad

Si la escena del menú principal contiene el GameManager y el jugador vuelve al menú, se creará una segunda instancia. El patrón Singleton de la Guía 7 (if Instance != null, Destroy gameObject) previene esto.

Los objetos con DontDestroyOnLoad se mueven a una escena especial llamada “DontDestroyOnLoad” visible en la Hierarchy durante Play.

1.5 PlayerPrefs: Persistencia Simple

PlayerPrefs permite guardar datos simples (enteros, floats, strings) que persisten entre sesiones del juego. Es ideal para high scores, configuraciones de volumen, o preferencias del jugador. Los datos se almacenan en el registro del sistema (Windows) o en archivos plist (macOS).

```
C# — PlayerPrefs
// Guardar
PlayerPrefs.SetInt("HighScore", 1500);
PlayerPrefs.Save();  // Forzar escritura a disco

// Leer (con valor por defecto si no existe)
int highScore = PlayerPrefs.GetInt("HighScore", 0);

// Eliminar
PlayerPrefs.DeleteKey("HighScore");
```

```
PlayerPrefs.DeleteAll();  // Borrar todo
```

2. Desarrollo Paso a Paso

Continuar con el proyecto **PlatformerGame** de las Guías 5–7. Este paso final transforma el prototipo en un juego completo y jugable.

1 Planificar la Estructura de Escenas

El juego tendrá al menos tres escenas:

Escena	Archivo	Contenido
Menú Principal	`MainMenu.unity`	Título del juego, botón Jugar, High Score, botón Salir
Nivel 1	`Level_01.unity`	Nivel de la Guía 5–7 con HUD (vidas, monedas)
Nivel 2	`Level_02.unity`	Segundo nivel con mayor dificultad

1.1 Registrar escenas en Build Profiles

5. Guardar la escena actual Level_01 si no está guardada.
6. Crear una nueva escena vacía: File > New Scene > Basic 2D (Built-in). Guardar como Assets/Scenes/MainMenu.unity.
7. Duplicar Level_01 y guardar como Assets/Scenes/Level_02.unity (se editará después).
8. Abrir Build Profiles: File > Build Profiles.
9. En Scene List, agregar las tres escenas en este orden:
   - Índice 0: MainMenu
   - Índice 1: Level_01
   - Índice 2: Level_02

Build Profiles en Unity 6

Unity 6 reemplazó la ventana Build Settings por Build Profiles. El flujo es similar: se agregan las escenas y se asigna un índice. El orden importa: la escena con índice 0 es la que se carga al iniciar el juego.

2 Refactorizar el GameManager con DontDestroyOnLoad

Actualizar el GameManager de la Guía 7 para persistir entre escenas, gestionar el flujo del juego y guardar el high score:

```
C# — GameManager.cs (refactorizado)
using UnityEngine;
using UnityEngine.SceneManagement;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    [Header("Game Settings")]
    [SerializeField] private int maxLives = 3;

    // Estado del juego
    private int currentScore;
    private int currentLives;
    private int highScore;

    // Propiedades públicas de solo lectura
    public int Score => currentScore;
    public int Lives => currentLives;
    public int HighScore => highScore;
    public int MaxLives => maxLives;

    // Eventos para notificar a la UI
    public System.Action<int> OnScoreChanged;
    public System.Action<int> OnLivesChanged;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
        DontDestroyOnLoad(gameObject);

        // Cargar high score guardado
        highScore = PlayerPrefs.GetInt(
            "HighScore", 0
        );

        InitializeGame();
    }

    public void InitializeGame()
    {
        currentLives = maxLives;
        currentScore = 0;
        OnScoreChanged?.Invoke(currentScore);
        OnLivesChanged?.Invoke(currentLives);
```

```
    }

    public void AddScore(int points)
    {
        currentScore += points;
        OnScoreChanged?.Invoke(currentScore);

        // Actualizar high score
        if (currentScore > highScore)
        {
            highScore = currentScore;
            PlayerPrefs.SetInt(
                "HighScore", highScore
            );
            PlayerPrefs.Save();
        }
    }

    public void TakeDamage()
    {
        currentLives--;
        OnLivesChanged?.Invoke(currentLives);

        if (currentLives <= 0)
        {
            GameOver();
        }
    }

    public void AddLife()
    {
        if (currentLives < maxLives)
        {
            currentLives++;
            OnLivesChanged?.Invoke(currentLives);
        }
    }

    private void GameOver()
    {
        // Cargar pantalla de Game Over
        // (implementada como panel en la UI)
        UIManager uiManager =
            FindFirstObjectByType<UIManager>();
        if (uiManager != null)
        {
            uiManager.ShowGameOver();
        }
    }

    public void RestartGame()
```

```
    {
        InitializeGame();
        SceneManager.LoadScene("Level_01");
    }

    public void LoadNextLevel()
    {
        int currentIndex =
            SceneManager.GetActiveScene().buildIndex;
        int nextIndex = currentIndex + 1;

        if (nextIndex < SceneManager.sceneCountInBuildSettings)
        {
            SceneManager.LoadScene(nextIndex);
        }
        else
        {
            // Último nivel completado: volver al menú
            LoadMainMenu();
        }
    }

    public void LoadMainMenu()
    {
        InitializeGame();
        SceneManager.LoadScene("MainMenu");
    }

    public void QuitGame()
    {
        PlayerPrefs.Save();
        #if UNITY_EDITOR
            UnityEditor.EditorApplication.isPlaying
                = false;
        #else
            Application.Quit();
        #endif
    }
}
```

Eventos (System.Action)

Los eventos OnScoreChanged y OnLivesChanged permiten que la UI se actualice automáticamente cuando cambian los valores, sin que el GameManager necesite conocer la UI directamente.

Este patrón Observer desacopla la lógica del juego de la presentación: el GameManager no hace referencia a ningún componente de UI.

FindFirstObjectByType<T>() es la API de Unity 6 que reemplaza a FindObjectOfType<T>().

3 Crear el HUD (Heads-Up Display)

Construir la interfaz en pantalla que muestra vidas y puntaje durante el juego:

3.1 Crear el Canvas del HUD

10. Abrir la escena Level_01.
11. En la Hierarchy: clic derecho > UI > Canvas. Renombrar a HUD_Canvas.
12. En el Canvas Scaler del Inspector:
    - UI Scale Mode: Scale With Screen Size
    - Reference Resolution: 1920 x 1080
    - Match: 0.5

3.2 Elemento de puntaje

13. Clic derecho sobre HUD_Canvas > UI > Text - TextMeshPro.
14. Si aparece el diálogo de TMP Essential Resources, hacer clic en Import.
15. Renombrar a ScoreText.
16. En el RectTransform, anclar a la esquina superior izquierda (Anchor Presets: Top-Left).
17. Posición: Pos X: 150, Pos Y: -50.
18. Texto inicial: “Monedas: 0”
19. Font Size: 36. Color: blanco. Outline: activar con color oscuro para legibilidad.

3.3 Indicador de vidas

Hay dos enfoques comunes. Se implementará el de texto por simplicidad:

Opción A (texto): Un TextMeshPro que muestra “Vidas: 3”. Simple y funcional.

Opción B (iconos): Tres imágenes de corazón que se desactivan al perder vidas. Más visual (ejercicio propuesto).

20. Crear otro Text - TextMeshPro hijo de HUD_Canvas. Renombrar a LivesText.
21. Anclar a la esquina superior derecha (Top-Right).
22. Posición: Pos X: -150, Pos Y: -50.
23. Texto inicial: “Vidas: 3”. Font Size: 36. Alignment: Right.

4 Crear el UIManager

Script que conecta el GameManager con los elementos de UI:

```
C# — UIManager.cs
using UnityEngine;
```

```
using TMPro;

public class UIManager : MonoBehaviour
{
    [Header("HUD")]
    [SerializeField] private TMP_Text scoreText;
    [SerializeField] private TMP_Text livesText;

    [Header("Panels")]
    [SerializeField] private GameObject pausePanel;
    [SerializeField] private GameObject gameOverPanel;

    private void Start()
    {
        // Suscribirse a los eventos del GameManager
        if (GameManager.Instance != null)
        {
            GameManager.Instance.OnScoreChanged +=
                UpdateScore;
            GameManager.Instance.OnLivesChanged +=
                UpdateLives;

            // Inicializar con valores actuales
            UpdateScore(GameManager.Instance.Score);
            UpdateLives(GameManager.Instance.Lives);
        }

        // Asegurar que los paneles estén ocultos
        if (pausePanel != null)
            pausePanel.SetActive(false);
        if (gameOverPanel != null)
            gameOverPanel.SetActive(false);
    }

    private void Update()
    {
        // Pausa con Escape
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            TogglePause();
        }
    }

    private void UpdateScore(int score)
    {
        if (scoreText != null)
            scoreText.text = $"Monedas: {score}";
    }

    private void UpdateLives(int lives)
```

```
    {
        if (livesText != null)
            livesText.text = $"Vidas: {lives}";
    }

    // ── Pausa ──
    public void TogglePause()
    {
        if (pausePanel == null) return;

        bool isPaused = pausePanel.activeSelf;
        pausePanel.SetActive(!isPaused);
        Time.timeScale = isPaused ? 1f : 0f;
    }

    public void ResumeGame()
    {
        if (pausePanel != null)
            pausePanel.SetActive(false);
        Time.timeScale = 1f;
    }

    // ── Game Over ──
    public void ShowGameOver()
    {
        if (gameOverPanel != null)
        {
            gameOverPanel.SetActive(true);
            Time.timeScale = 0f;
        }
    }

    // ── Botones ──
    public void OnRestartButton()
    {
        Time.timeScale = 1f;
        GameManager.Instance.RestartGame();
    }

    public void OnMainMenuButton()
    {
        Time.timeScale = 1f;
        GameManager.Instance.LoadMainMenu();
    }

    public void OnQuitButton()
    {
        GameManager.Instance.QuitGame();
    }
```

```
    private void OnDestroy()
    {
        // Desuscribirse para evitar memory leaks
        if (GameManager.Instance != null)
        {
            GameManager.Instance.OnScoreChanged -=
                UpdateScore;
            GameManager.Instance.OnLivesChanged -=
                UpdateLives;
        }
    }
}
```

4.1 Configurar el UIManager en la escena

24. Crear un GameObject vacío llamado UIManager en la Hierarchy.
25. Agregar el script UIManager.
26. Arrastrar ScoreText y LivesText a los campos correspondientes en el Inspector.

OnDestroy y desuscripción

Es fundamental desuscribirse de los eventos en OnDestroy. Si la UI se destruye al cambiar de escena pero el GameManager persiste (DontDestroyOnLoad), los eventos seguirían apuntando a objetos destruidos, causando errores NullReferenceException.

5 Crear los Paneles de Pausa y Game Over

Agregar paneles de UI que se muestran/ocultan según el estado del juego:

5.1 Panel de Pausa

27. Clic derecho sobre HUD_Canvas > UI > Panel. Renombrar a PausePanel.
28. Configurar el Image del panel: color negro con alpha 180/255 (semi-transparente).
29. Agregar elementos hijos:
    - Text - TextMeshPro: “PAUSA”, centrado, font size 72, color blanco.
    - Button - TextMeshPro: “Continuar”. En el evento OnClick, asignar UIManager.ResumeGame.
    - Button - TextMeshPro: “Menú Principal”. OnClick → UIManager.OnMainMenuButton.
30. Desactivar el PausePanel (desmarcar el checkbox en el Inspector) para que inicie oculto.

5.2 Panel de Game Over

31. Crear otro Panel hijo de HUD_Canvas. Renombrar a GameOverPanel.
32. Fondo: color rojo oscuro con alpha 200/255.
33. Agregar elementos hijos:

- Text - TextMeshPro: “GAME OVER”, font size 80, color blanco.
- Text - TextMeshPro: “Puntaje: 0” (se actualizará dinámicamente).
- Button: “Reintentar”. OnClick → UIManager.OnRestartButton.
- Button: “Menú Principal”. OnClick → UIManager.OnMainMenuButton.

34. Desactivar el GameOverPanel.

35. Asignar ambos paneles en el Inspector del UIManager (campos pausePanel y gameOverPanel).

Time.timeScale = 0

Establecer Time.timeScale en 0 congela toda la física y las animaciones. Los botones de UI siguen funcionando porque responden a eventos del EventSystem, no al tiempo del juego.

Es fundamental restaurar Time.timeScale = 1 antes de cambiar de escena. Si no, la siguiente escena también estará “congelada”.

6 Construir el Menú Principal

Crear la pantalla de inicio del juego:

6.1 Diseñar la escena MainMenu

36. Abrir la escena MainMenu.unity.
37. Crear un Canvas con Canvas Scaler configurado igual que el HUD (Scale With Screen Size, 1920x1080, Match 0.5).
38. Cambiar el color de fondo de la cámara a un color acorde al juego.

6.2 Elementos del menú

39. Text - TextMeshPro: Título del juego (ej: “Platformer Adventure”), font size 80, centrado en la parte superior.
40. Text - TextMeshPro: “Record: 0” para mostrar el high score. Se actualizará con PlayerPrefs.
41. Button: “Jugar” — el botón principal.
42. Button: “Salir” — cierra la aplicación.

6.3 Script MainMenuManager

Crear Assets/Scripts/MainMenuManager.cs:

```
C# — MainMenuManager.cs
```

```
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;

public class MainMenuManager : MonoBehaviour
{
    [SerializeField] private TMP_Text highScoreText;

    private void Start()
    {
        // Mostrar high score guardado
        int highScore = PlayerPrefs.GetInt(
            "HighScore", 0
        );
        if (highScoreText != null)
        {
            highScoreText.text =
                $"Record: {highScore}";
        }

        // Asegurar que el tiempo esté normal
        Time.timeScale = 1f;
    }

    public void OnPlayButton()
    {
        // Inicializar el juego si el
        // GameManager ya existe
        if (GameManager.Instance != null)
        {
            GameManager.Instance.InitializeGame();
        }

        SceneManager.LoadScene("Level_01");
    }

    public void OnQuitButton()
    {
        PlayerPrefs.Save();
        #if UNITY_EDITOR
            UnityEditor.EditorApplication.isPlaying
                = false;
        #else
            Application.Quit();
        #endif
    }
}
```

43. Crear un GameObject vacío MenuManager y asignar el script.
44. Conectar los botones a los métodos: Jugar → OnPlayButton, Salir → OnQuitButton.

45. Asignar el texto de high score al campo del Inspector.

6.4 Colocar el GameManager en MainMenu

46. Crear un GameObject vacío GameManager en la escena MainMenu.
47. Agregar el script GameManager.cs. Como tiene DontDestroyOnLoad, persistirá al cargar Level_01.
48. Si el GameManager ya existía en Level_01, eliminarlo de esa escena (ahora vive en MainMenu y sobrevive entre escenas).

Una sola fuente de verdad

El GameManager solo debe existir en la escena MainMenu (la primera en cargarse). Gracias al Singleton + DontDestroyOnLoad, persistirá al cargar cualquier nivel. Si se coloca en múltiples escenas, el Singleton destruirá las copias duplicadas, pero es más limpio tenerlo en una sola.

7 Implementar Sistema de Checkpoints

Actualizar el punto de reaparición dinámicamente cuando el jugador alcanza un checkpoint:

```
C# — Checkpoint.cs
using UnityEngine;

public class Checkpoint : MonoBehaviour
{
    [SerializeField] private SpriteRenderer flagSprite;
    [SerializeField] private Color activatedColor
        = Color.green;

    private bool isActivated;
    private Color originalColor;

    private void Start()
    {
        if (flagSprite != null)
            originalColor = flagSprite.color;
    }

    private void OnTriggerEnter2D(
        Collider2D collision)
    {
        if (isActivated) return;
        if (!collision.CompareTag("Player")) return;

        isActivated = true;
```

```
        // Actualizar el respawn en la KillZone
        KillZone killZone =
            FindFirstObjectByType<KillZone>();
        if (killZone != null)
        {
            killZone.SetRespawnPoint(
                transform.position
            );
        }

        // Feedback visual
        if (flagSprite != null)
        {
            flagSprite.color = activatedColor;
        }

        Debug.Log($"Checkpoint activado en " +
            $"{transform.position}");
    }
}
```

7.1 Actualizar KillZone

Agregar un método público a KillZone.cs para cambiar el punto de reaparición:

```
C# — Método adicional en KillZone.cs
// Agregar este método a KillZone.cs
public void SetRespawnPoint(Vector3 newPosition)
{
    // Crear un nuevo punto o mover el existente
    respawnPoint.position = newPosition
        + Vector3.up * 1f;  // Ligeramente arriba
}
```

7.2 Colocar checkpoints en el nivel

49. Crear un GameObject con un sprite de bandera o poste.
50. Agregar un BoxCollider2D marcado como Is Trigger, un poco más ancho que el sprite.
51. Agregar el script Checkpoint. Asignar el SpriteRenderer al campo flagSprite.
52. Crear prefab y distribuir 2–3 checkpoints a lo largo de Level_01.
53. Repetir para Level_02.

8 Crear la Transición entre Niveles

Implementar el paso de Level_01 a Level_02 cuando el jugador llega al final:

8.1 Script LevelEnd

```
C# — LevelEnd.cs
using UnityEngine;

public class LevelEnd : MonoBehaviour
{
    [SerializeField] private string nextSceneName;

    private void OnTriggerEnter2D(
        Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            if (GameManager.Instance != null)
            {
                GameManager.Instance.LoadNextLevel();
            }
        }
    }
}
```

54. Crear un GameObject vacío llamado LevelEnd al final del nivel.
55. Agregar un BoxCollider2D marcado como Is Trigger (zona amplia para que el jugador la active al pasar).
56. Agregar el script LevelEnd.
57. Colocar un sprite visual (puerta, portal, bandera de meta) para indicar el final.

8.2 Preparar Level_02

58. Abrir la escena Level_02.
59. Modificar el nivel: cambiar el diseño del tilemap, agregar más enemigos, plataformas más difíciles.
60. Asegurarse de que Level_02 tenga: HUD_Canvas con UIManager, KillZone, checkpoints, y un LevelEnd (que al ser el último nivel, llevará al menú principal).

> **No duplicar el GameManager en Level_02**
> El GameManager ya existe gracias a DontDestroyOnLoad desde MainMenu. Si se duplica en Level_02, el Singleton destruirá la copia, pero puede causar un frame de confusión. Mantenerlo solo en MainMenu.

8.3 Flujo completo del juego

```
Diagrama de flujo del juego
MainMenu  ──[Jugar]──>  Level_01
                            |
                      [Llegar al final]
                            |
                            v
                        Level_02
                            |
                      [Llegar al final]
                            |
                            v
                       MainMenu
                    (con High Score)

En cualquier nivel:
  [Game Over] → Panel Game Over
     ├─ [Reintentar] → Level_01
     └─ [Menú]       → MainMenu

  [Escape]    → Panel Pausa
     ├─ [Continuar] → Reanudar
     └─ [Menú]      → MainMenu
```

3. Ejercicios Propuestos

Ejercicio 1: Vidas como Iconos de Corazón

Reemplazar el texto “Vidas: 3” por una fila de 3 imágenes de corazón en el HUD. Al perder una vida, el corazón correspondiente cambia a una versión grisácea (o se desactiva). Usar un arreglo de Image en el UIManager y desactivar/activar según currentLives. Este enfoque es más visual y profesional.

Ejercicio 2: Transición con Fade (Fundido a Negro)

Implementar un efecto de fundido a negro (fade out) al cambiar de nivel y un fundido desde negro (fade in) al cargar el nuevo nivel. Crear un Panel negro de pantalla completa en el Canvas y animar su alpha con una Coroutine (de 0 a 1 para fade out, de 1 a 0 para fade in). Invocar SceneManager.LoadScene dentro de la Coroutine después del fade out.

Ejercicio 3: Pantalla de Victoria

Crear una pantalla de victoria que se muestre al completar el último nivel. Debe mostrar el puntaje final, el high score, un mensaje de felicitación, y botones para volver al menú o reiniciar desde el primer nivel. Agregar la lógica en GameManager.LoadNextLevel para detectar cuándo se completa el último nivel.

4. Bibliografía y Recursos

- *Unity Technologies.* Unity Manual — Canvas. https://docs.unity3d.com/Manual/UICanvas.html
- *Unity Technologies.* Unity Manual — TextMeshPro. https://docs.unity3d.com/Packages/com.unity.textmeshpro@4.0/manual/index.html
- *Unity Technologies.* Unity Manual — SceneManager. https://docs.unity3d.com/ScriptReference/SceneManagement.SceneManager.html
- *Unity Technologies.* Unity Manual — DontDestroyOnLoad. https://docs.unity3d.com/ScriptReference/Object.DontDestroyOnLoad.html
- *Unity Technologies.* Unity Manual — PlayerPrefs. https://docs.unity3d.com/ScriptReference/PlayerPrefs.html
- *Hocking, J.* Unity in Action, Third Edition. Manning Publications, 2022.